```
"""Utility functions."""

from __future__ import division

import random, math


def rand_select(probs):
    """Return key with probability probs[key] (normalised).

    If probs is a sequence, return one of range(len(probs)).

    """
    # How do I distinguish mappings from sequences?
    # The best way I found so far...
    if hasattr(probs, 'keys'):
        keys = probs.keys()
        values = probs.values()
    else:
        keys = range(len(probs))
        values = probs

    from operator import add
    sum = reduce(add, values)

    r = random.random() * sum
    for key in keys:
        r -= probs[key]
        if r < 0:
            return key
    return key                      # last is catch-all (for rounding errors)


class Statistics(object):
    """Collector of values that tracks the mean, deviation, min and max."""
    def __init__(self, *args):
        """Clear and then submit given values/stats (if any)."""
        self.cnt = self.sum = self.sqsum = 0
        self.min = self.max = None
        for a in args:
            self.submit(a)
    def submit(self, x):
        """Add a single number or a whole stat object."""
        if isinstance(x, Statistics):
            self.cnt += x.cnt
            self.sum += x.sum
            self.sqsum += x.sqsum
            if self.min is None or self.min > x.min:
                self.min = x.min
            if self.max is None or self.max < x.max:
                self.max = x.max
        else:
            self.cnt += 1
            self.sum += x
            self.sqsum += x**2
            if self.min is None or self.min > x:
                self.min = x
            if self.max is None or self.max < x:
                self.max = x
    def mean(self):
        if self.cnt > 1:
            return self.sum / self.cnt
        else:
            return 0.
    def rms(self):
        if self.cnt > 1:
            return math.sqrt(self.sqsum / self.cnt)
        else:
            return 0.
    def deviation(self):
```

```
        if self.cnt > 1:
            return math.sqrt((self.sqsum - self.sum**2 / self.cnt) /
                             (self.cnt - 1))
        else:
            return 0. # None could be better but would cause exceptions.
    __float__ = mean
    def __str__(self):
        return "(%s < %.3f +- %.3f < %s x %s)" % (
            self.min, self.mean(), math.sqrt(self.deviation()),
            self.max, self.cnt)


def smap(spec, obj):
    """smap(spec, obj) - destructuring map; useful for debugging data structures...

    smap(func, obj) -> func(obj)
    smap([spec0, spec1, ...], seq) -> [smap(spec1, obj[0]), smap(spec1, obj[1]), ...]
    smap([spec], seq) -> [smap(spec, obj[0]), smap(spec, obj[1]), ...]
    smap((...), seq) -> tuple(smap(..., seq))
    smap({kspec, vspec}, {key0: val0, key1: val1, ...}) ->
      {smap(kspec, key0): smap(vspec, val0), ...}

    """
    if callable(spec):
        return spec(obj)
    elif spec is None:
        return obj
    elif isinstance(spec, tuple):
        return tuple(smap(list(spec), obj))
    elif isinstance(spec, list):
        if len(spec) == 1:
            return [smap(spec[0], x) for x in obj]
        else:
            return [smap(spec[i], obj[i]) for i in range(len(spec))]
    elif isinstance(spec, dict):
        kspec, vspec = spec.items()[0]
        return dict([(smap(kspec, key), smap(vspec, val))
                     for key, val in obj.items()])
    raise Exception("Unknown spec type")
```